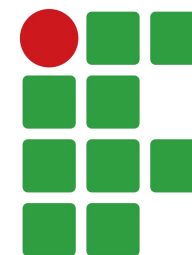


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

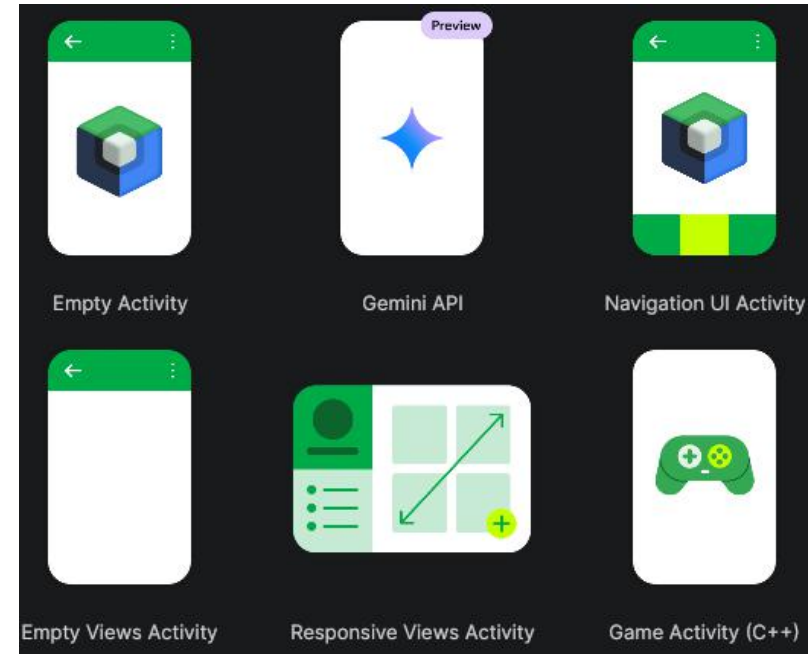
Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco

Activity e ciclo de vida

- A Activity representa uma tela ou ponto de interação da aplicação e é responsável por gerenciar sua interface e interações com o usuário.
- Diferentemente de aplicações tradicionais com um método `main()`, o Android gerencia o ciclo de vida das Activities, invocando métodos de callback em diferentes momentos.

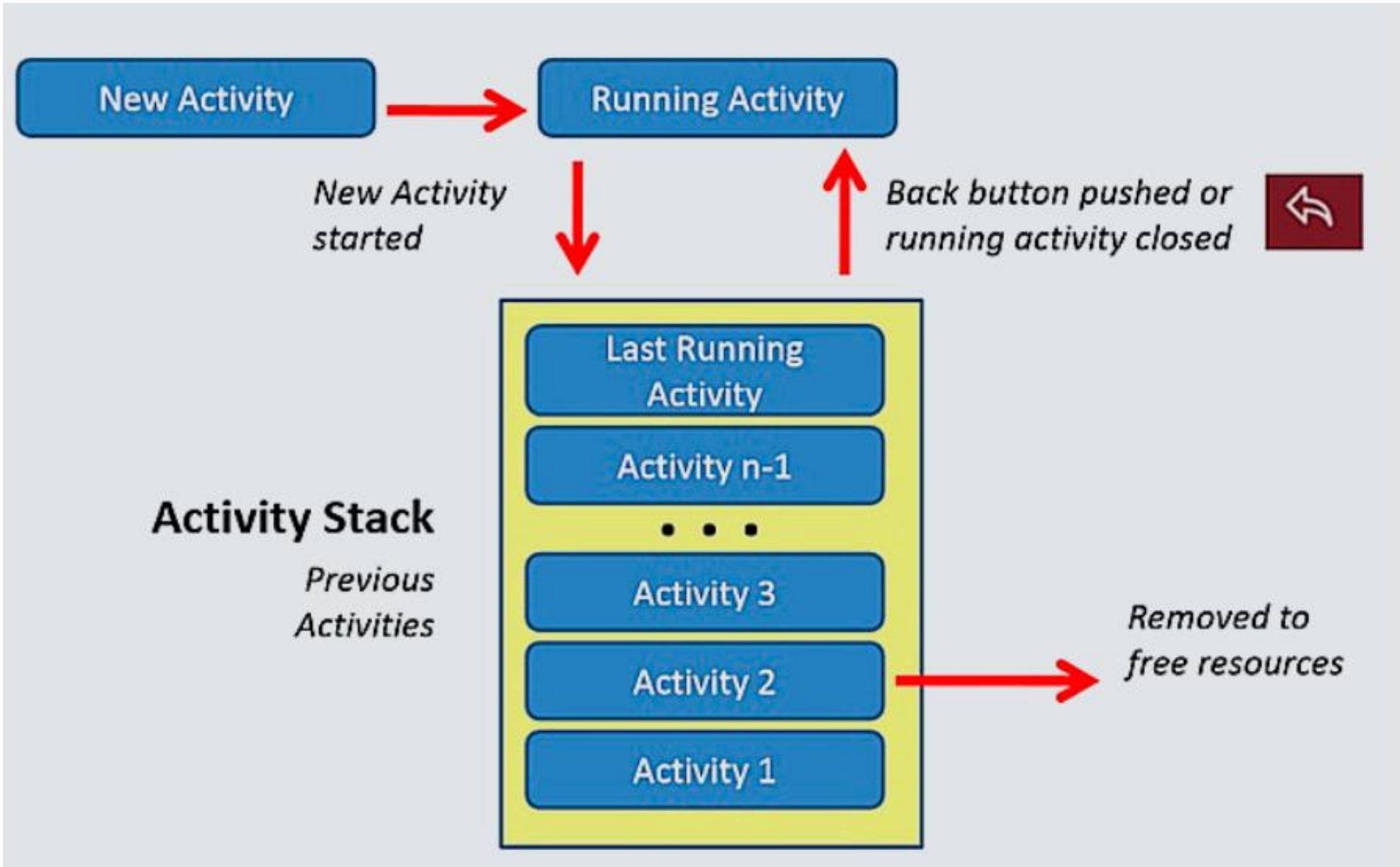


Activity e ciclo de vida

- As Activities da aplicação são declaradas no arquivo AndroidManifest.xml.
- A declaração é feita utilizando o elemento <activity>, dentro de <application>.
- O Android utiliza essa informação para conhecer e gerenciar os componentes da aplicação.

```
<manifest ... >
  <application ... >
    <activity android:name=".ExampleActivity" />
    ...
  </application ... >
  ...
</manifest >
```

Activity e ciclo de vida



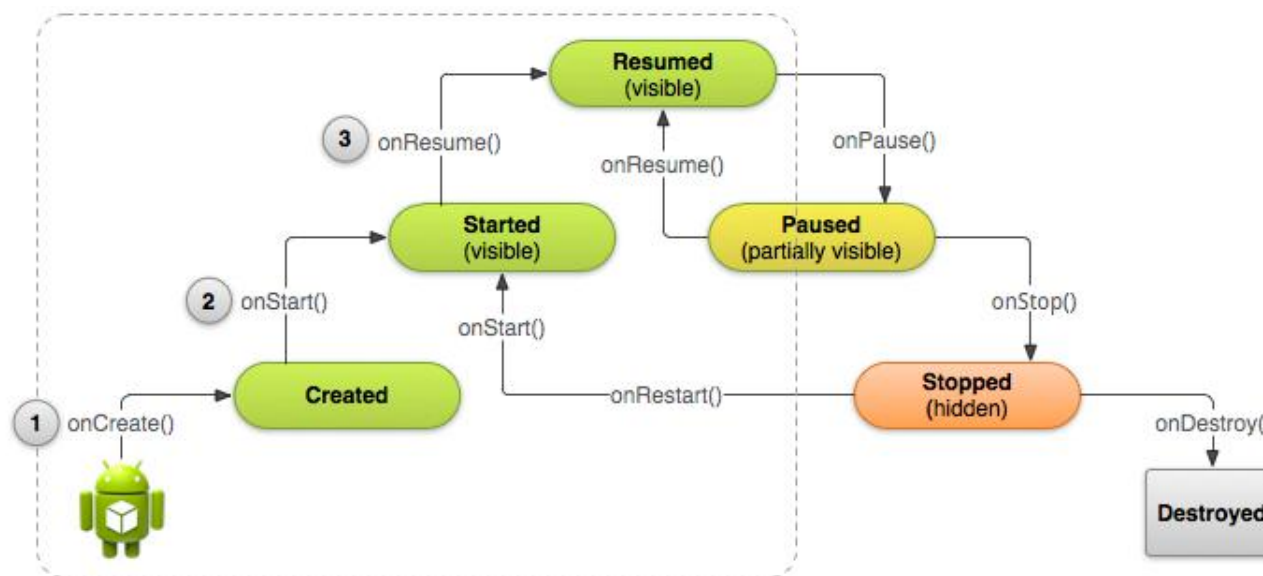
Activity e ciclo de vida

- O Android gerencia as Activities e sua navegação, mantendo uma pilha de Activities (back stack).
- Ao iniciar uma nova Activity, ela normalmente é colocada no topo da pilha.
- A Activity anterior permanece na pilha e pode voltar a ficar visível quando a nova Activity for encerrada.
- Ao pressionar Voltar, a Activity do topo é removida e a anterior é retomada.
- Em situações de pouca memória, o sistema pode destruir processos que não estejam em uso, podendo posteriormente recriar suas Activities.

Activity e ciclo de vida

- Durante seu ciclo de vida, uma Activity pode passar por diferentes estados:
 - Criada: a Activity está sendo inicializada.
 - Iniciada: está visível para o usuário.
 - Em primeiro plano: está visível e recebendo interação.
 - Pausada: perdeu o foco, mas ainda pode estar parcialmente visível.
 - Parada: não está mais visível ao usuário.
 - Destruída: a instância da Activity foi encerrada.
- O sistema Android controla as transições entre esses estados.
- O desenvolvedor deve implementar os callbacks apropriados para cada etapa do ciclo de vida.

Activity e ciclo de vida



Entire lifetime

Tempo de vida completo

Ocorre desde a primeira chamada ao método **onCreate()** até a chamada do método **onDestroy()**, os quais são executados apenas uma única vez durante o ciclo de vida da Activity.

Visible lifetime

Tempo de vida visível

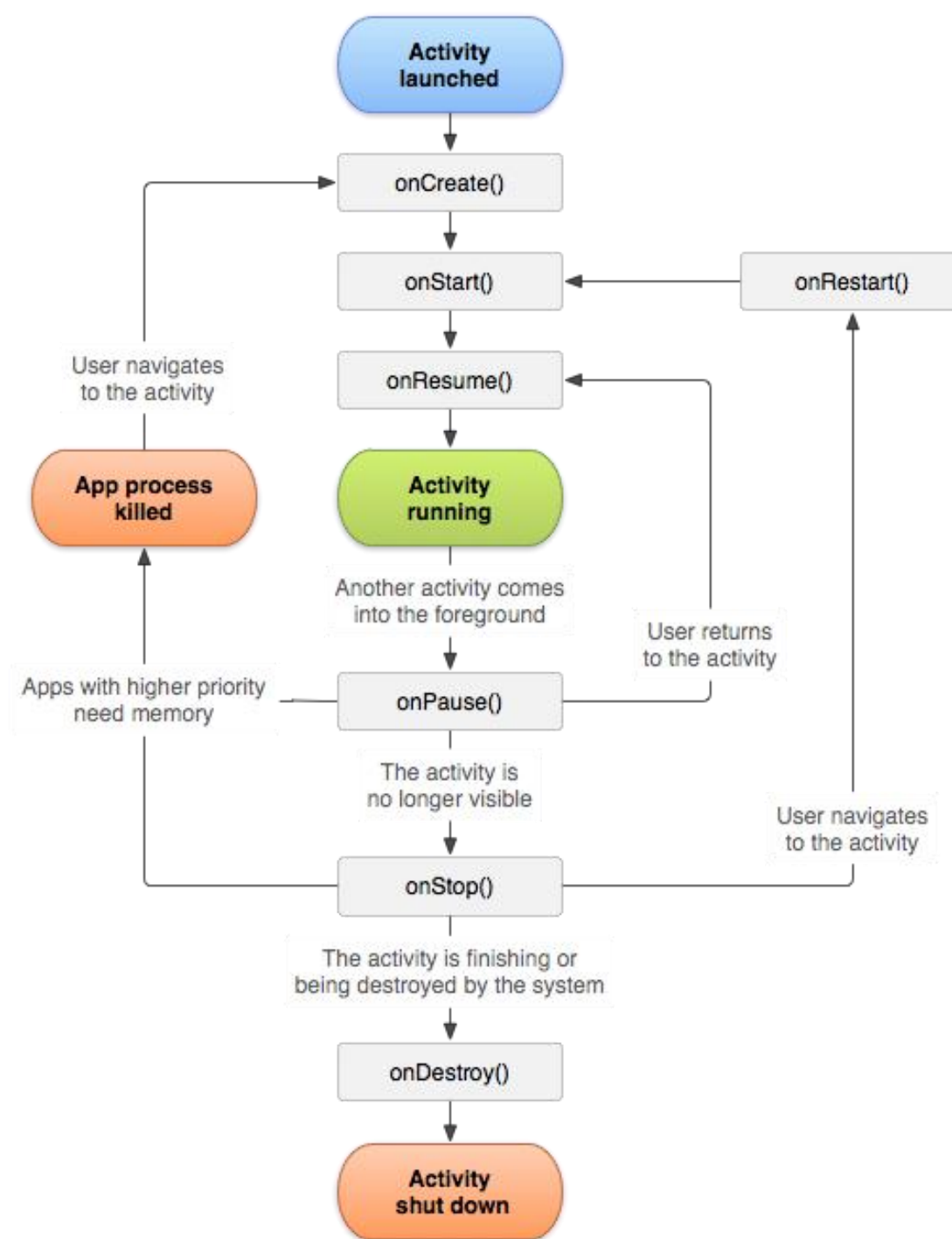
Ocorre entre uma chamada do método **onStart()** e a chamada correspondente do método **onStop()**. A Activity pode estar no topo da pilha (visível para o usuário) ou em segundo plano.

Foreground lifetime

Tempo de vida no topo da pilha

Ocorre entre uma chamada do método **onResume()** e a chamada correspondente do método **onPause()**. A Activity está no topo da pilha e interagindo com o usuário.

Activity e ciclo de vida



Exemplo: recebendo uma ligação durante um jogo

- Imagine que o usuário está jogando e recebe uma ligação telefônica:
 - O Android precisa interromper temporariamente a interação com o jogo.
 - A Activity do jogo passa por mudanças no seu ciclo de vida.
 - O sistema invoca os callbacks correspondentes a essas mudanças.
 - O desenvolvedor pode implementar esses callbacks para salvar o estado e liberar ou recuperar recursos.
 - Quando o usuário retorna ao jogo, a Activity pode retomar sua execução.

Activity e ciclo de vida

- Activity em segundo plano:
 - Quando uma Activity deixa de estar visível, seu processo pode permanecer em memória.
 - Em situações de pressão de memória, o Android pode encerrar o processo para liberar recursos.
 - Caso o processo seja encerrado, o Android poderá recriar a Activity posteriormente, sendo necessário restaurar seu estado.

Activity e ciclo de vida

- Uma boa implementação do ciclo de vida ajuda a aplicação a:
 - Lidar corretamente com interrupções, como chamadas telefônicas ou mudança para outro aplicativo.
 - Liberar recursos quando a aplicação não está em uso.
 - Preservar o estado e o progresso do usuário.
 - Tratar mudanças de configuração, como a rotação da tela.

Logcat e classe Log

- A classe Log permite registrar mensagens de depuração que podem ser visualizadas no Logcat do Android Studio.
- Os principais níveis de log, em ordem decrescente de prioridade, são:
 - `Log.e(String, String)` (erro)
 - `Log.w(String, String)` (aviso)
 - `Log.i(String, String)` (informações)
 - `Log.d(String, String)` (depuração)
 - `Log.v(String, String)` (detalhado)

Prática 1: Analisando o Ciclo de Vida da Activity com Logs

```
class MainActivity : ComponentActivity() {  
    10 Usages  
    private lateinit var binding: ActivityMainBinding  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
  
        createListeners()  
    }  
}
```



Prática 1: Analisando o Ciclo de Vida da Activity com Logs

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    binding = ActivityMainBinding.inflate(layoutInflater)  
    setContentView(binding.root)  
  
    createListeners()  
}  
  
override fun onStart() {  
    super.onStart()  
}  
  
override fun onRestart() {  
    super.onRestart()  
}  
  
override fun onResume() {  
    super.onResume()  
}
```

```
override fun onPause() {  
    super.onPause()  
}  
  
override fun onStop() {  
    super.onStop()  
}  
  
override fun onDestroy() {  
    super.onDestroy()  
}
```

Prática 1: Analisando o Ciclo de Vida da Activity com Logs

```
import android.util.Log
```

```
companion object{ const val TAG = "DEBUG LIFECYCLE"}

override fun onCreate(savedInstanceState: Bundle?) {
    Log.d(TAG, msg = "0 método onCreate foi invocado")
    super.onCreate(savedInstanceState)
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    createListeners()
}

override fun onStart() {
    Log.d(TAG, msg = "0 método onStart foi invocado")
    super.onStart()
}

override fun onRestart() {
    Log.d(TAG, msg = "0 método onRestart foi invocado")
    super.onRestart()
}
```

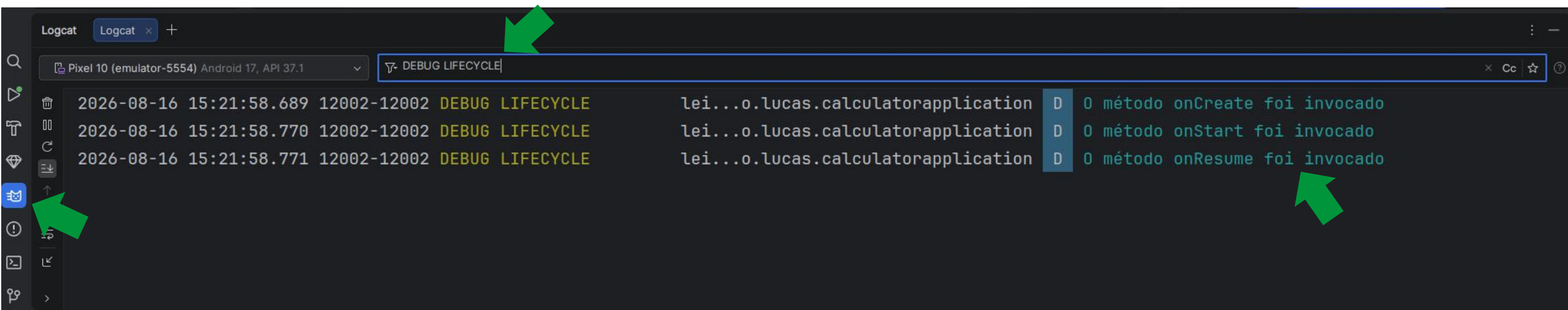
```
override fun onResume() {
    Log.d(TAG, msg = "0 método onResume foi invocado")
    super.onResume()
}

override fun onPause() {
    Log.d(TAG, msg = "0 método onPause foi invocado")
    super.onPause()
}

override fun onStop() {
    Log.d(TAG, msg = "0 método onStop foi invocado")
    super.onStop()
}

override fun onDestroy() {
    Log.d(TAG, msg = "0 método onDestroy foi invocado")
    super.onDestroy()
}
```

Prática 1: Analisando o Ciclo de Vida da Activity com Logs



Logcat Logcat x +

Pixel 10 (emulator-5554) Android 17, API 37.1

DEBUG LIFECYCLE

```
2026-08-16 15:21:58.689 12002-12002 DEBUG LIFECYCLE lei...o.lucas.calculatorapplication D 0 método onCreate foi invocado
2026-08-16 15:21:58.770 12002-12002 DEBUG LIFECYCLE lei...o.lucas.calculatorapplication D 0 método onStart foi invocado
2026-08-16 15:21:58.771 12002-12002 DEBUG LIFECYCLE lei...o.lucas.calculatorapplication D 0 método onResume foi invocado
```

Prática 1: Analisando o Ciclo de Vida da Activity com Logs

- Cenário 1: Sair e voltar
 - Execute o app.
 - Pressione Home.
 - Retorne ao app.
 - Observe os métodos no Logcat.
- Cenário 2: Remover das recentes
 - Pressione Home.
 - Abra os aplicativos recentes.
 - Remova a calculadora da lista.
 - Abra o app novamente.
 - Observe os métodos no Logcat.
- Cenário 3: Rotacionar
 - Preencha os campos da calculadora.
 - Gire a tela.
 - Observe os métodos no Logcat.
 - Verifique o que aconteceu com os dados.

Salvando e restaurando o estado da Activity

- Os métodos `onSaveInstanceState()` e `onRestoreInstanceState()` permitem preservar e restaurar o estado temporário de uma Activity, especialmente durante mudanças de configuração, como a rotação da tela.
 - `onSaveInstanceState()`: permite salvar informações importantes antes que a Activity possa ser destruída e recriada.
 - `onRestoreInstanceState()`: permite recuperar o estado salvo após a Activity ser recriada. É chamado depois de `onStart()` e somente quando existe um estado previamente salvo.

Salvando e restaurando o estado da Activity

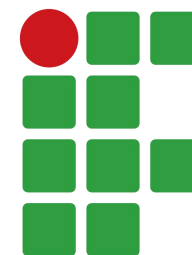
```
override fun onSaveInstanceState(outState: Bundle) {  
    super.onSaveInstanceState(outState)  
    outState.putString("etNumber1", binding.etNumber1.text.toString())  
    outState.putString("etNumber2", binding.etNumber2.text.toString())  
    outState.putString("tvResultNumber", binding.tvResultNumber.text.toString())  
}  
  
override fun onRestoreInstanceState(savedInstanceState: Bundle) {  
    super.onRestoreInstanceState(savedInstanceState)  
    binding.etNumber1.setText(savedInstanceState.getString(key = "etNumber1"))  
    binding.etNumber2.setText(savedInstanceState.getString(key = "etNumber2"))  
    binding.tvResultNumber.text = savedInstanceState.getString(key = "tvResultNumber")  
}
```

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco